TEAM PROJECT PRESENTATION

텍스트 RPG 세계관에서 하렘을 구축하는 방법



호감도 기반 성장, 히로인 중심 전투, 상성 시스템일 결합한
차별화된 텍스트 RPG 프로젝트

Team

모태솔로

우리 팀만의 5가지 차별점

전투 주체, 성장 방식, 전략 요소, 코드 구조, 결합도 관리 측면에서 기존 RPG와 차별화된 5가지 핵심 요소를 요약합니다.

#	차별점	기존 RPG	우리 프로젝트
01	전투 주체	플레이어가 직접 전투	히로인이 전투 수행 (명령 중심)
02	전략 요소	단순 스탯 위주의 힘 싸움	상성 시스템 적용
03	코드 구조	단일 파일 또는 기능 혼합 구조	기능별 폴더 구조 분리
04	UI 표현 방식	단순 텍스트 출력	델리게이트 기반 UI + 이미지형 콘솔 렌더링
05	객체 관리 방식	Raw Pointer 사용	shared_ptr + unique_ptr 사용

히로인 배틀 시스템

플레이어는 지시자 역할을 수행하고 히로인이 실제 전투를 담당하며,
연애 요소인 호감도가 전투 결과에 직접적인 영향을 미칩니다.

전투 지시자 (Commander)

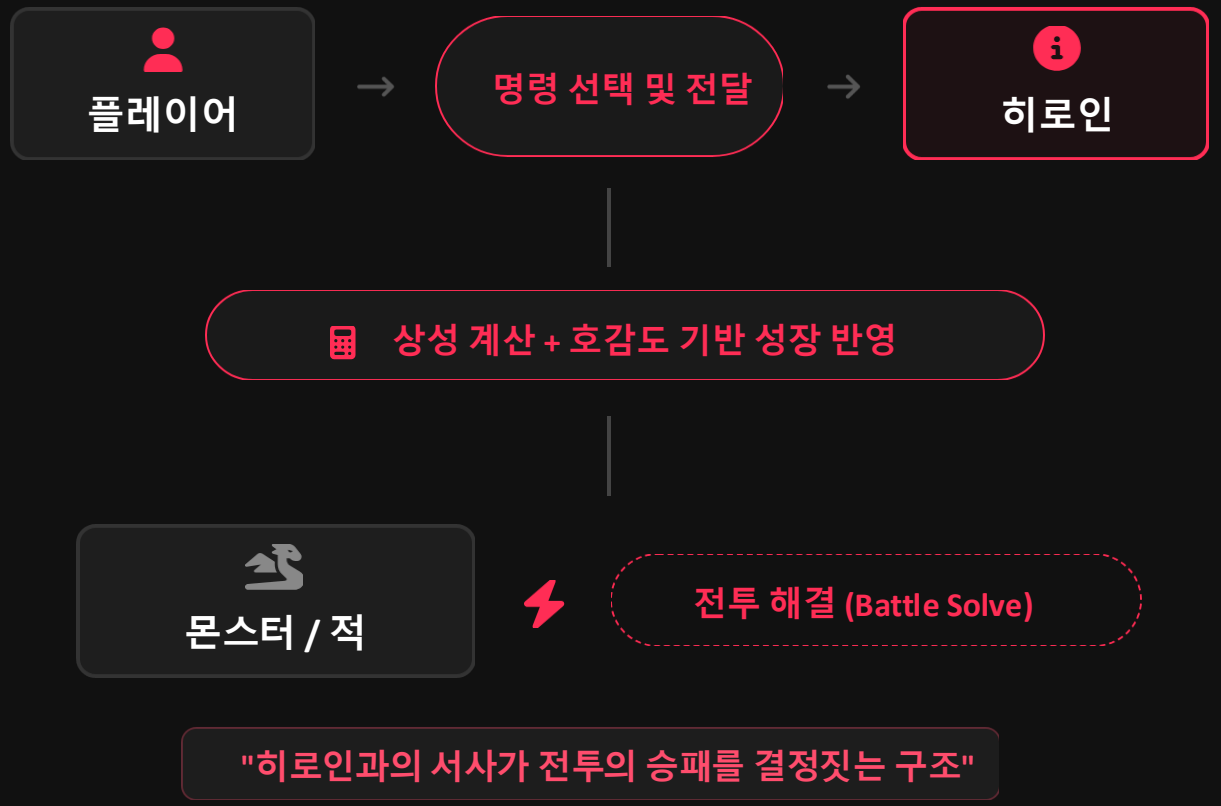
플레이어는 직접 공격하지 않고 전략적 명령과 아이템 사용에 집중합니다

실질적 전투 주체

히로인이 데미지를 계산하고 아이템을 사용하여 적을 격퇴하는 핵심 역할을 수행합니다.

호감도 기반 성장 시스템

경험치 대신 호감도를 누적해 레벨업이 이루어지며 성장한 레벨에 따라 최대 체력과 공격력이 상승하도록 설계



상성 시스템

히로인의 스타일과 적의 타입 사이에 상성 관계를 적용하여, 전략적 선택에 따라 최종 데미지 배율이 달라지도록 설계하였습니다.

Advantage
상성 유리 × 1.5

Neutral
상성 중립 × 1.0

Disadvantage
상성 불리 × 0.5

히로인 스타일	강한 상대 (유리)	약한 상대 (불리)
HotGirl	GrassGirl 스타일 적	IceGirl 스타일 적
IceGirl	HotGirl 스타일 적	GrassGirl 스타일 적
GrassGirl	IceGirl 스타일 적	HotGirl 스타일 적

단순 수치 경쟁이 아니라, 상대 스타일에 맞는 히로인 선택이 실제 데미지 차이로 이어지는 전략 요소를 구현하였습니다.

UI 이미지 출력 및 델리게이트 구조

전투와 상태 변화 결과를 델리게이트로 전달하고, 콘솔 UI에서는 이미지 렌더링과 ANSI 컬러 로그 출력을 통해 시작적으로 표현했습니다.

이미지 기반 콘솔 UI

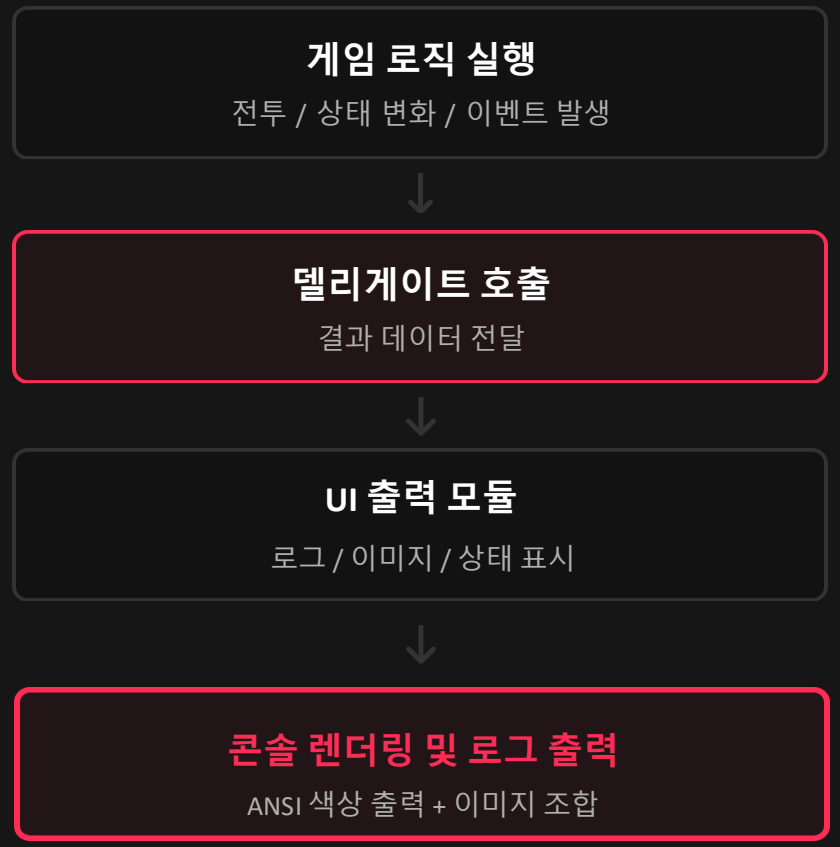
std_image를 활용해 캐릭터 이미지를 로드하고.
콘솔 환경에서도 시각적인 화면 구성이 가능하도록 구현

알파 블렌딩 렌더링

투명도(alpha)를 반영해 이미지와 배경을 자연스럽게 합성하고,
콘솔 화면 위에 오브젝트를 겹쳐 표현하는 구조

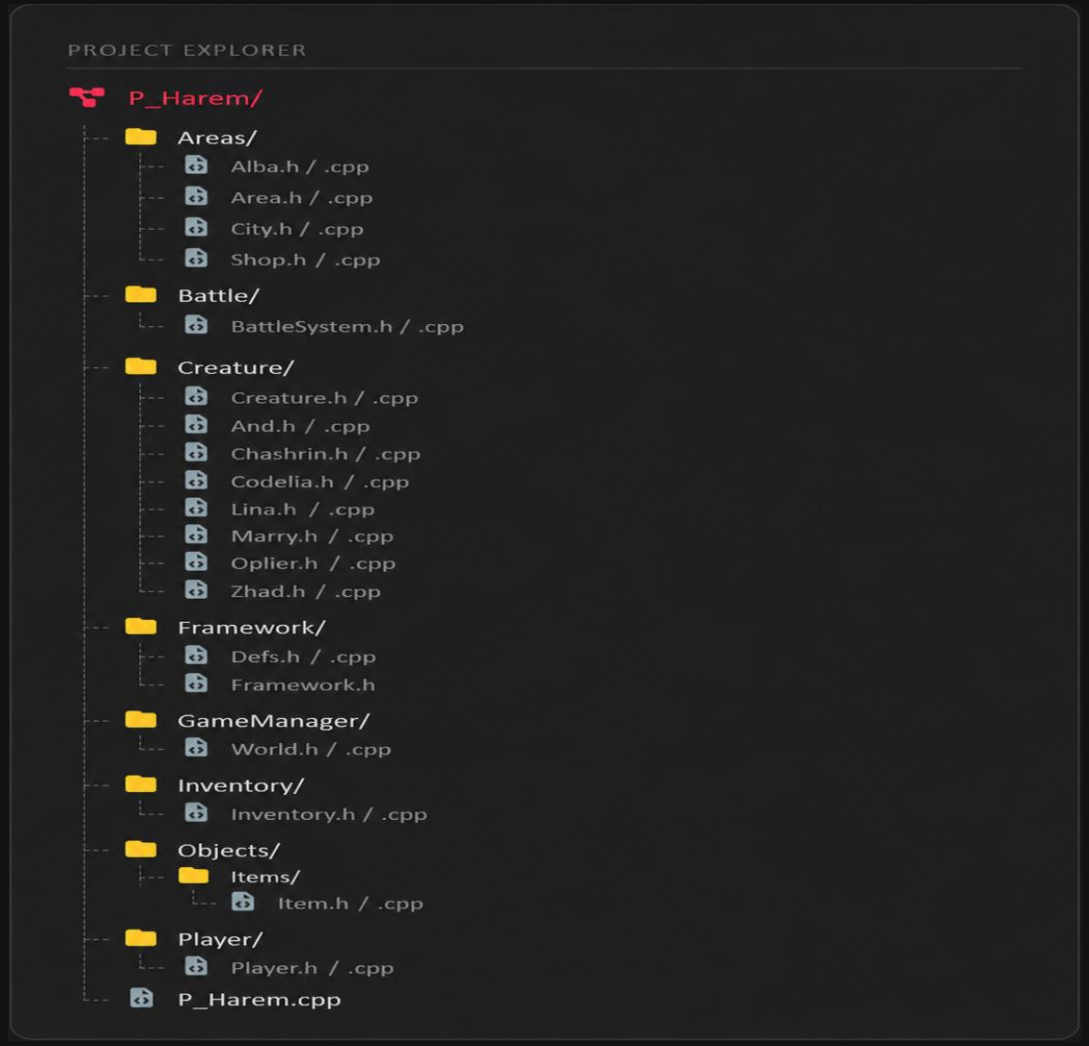
델리게이트 기반 로그 출력

전투 결과와 상태 변화를 델리게이트로 전달하여
로직과 출력 처리를 분리하고 UI 로그를 유연하게 관리



의존성 주입 + 객체 소유 구조

기능 단위 모듈 구조를 적용하여 팀원 간 역할 분담을 명확히 하고, 병렬 개발과 유지보수 효율성을 높였습니다.



기능 중심 모듈화

Areas, Battle, Creature, Framework, GameManger, Inventory, Objects, Player 등 기능 기준으로 폴더를 분리하여 각 시스템의 역할이 섞이지 않도록 구성하였습니다.

협업 구조 최적화

팀원별 담당 기능을 폴더 단위로 나누어 코드 충돌을 줄이고 동시에 작업하기 쉬운 협업 구조를 만들었습니다.

유지 보수 효율 향상

특정 기능 수정 시 관련 폴더와 파일만 집중해서 확인할 수 있어 디버깅과 기능 확장이 훨씬 수월해졌습니다.

델리게이트 기반 UI 와 Logic 분리

델리게이트를 활용해 전투 로직과 출력 처리를 분리함으로써 코드 결합도를 낮추고 유지보수와 확장성을 높였습니다.

UI와 Logic 분리

전투 시스템은 화면 출력 방식을 직접 알지 않고, 필요한 전투 결과 데이터만 델리게이트를 통해 전달합니다.

이벤트 기반 처리

공격, 피격, 전투 종료와 같은 이벤트가 발생했을 때만 연결된 출력 로직이 반응하도록 설계했습니다.

유지보수와 확장성

출력 방식이 바뀌더라도 전투 로직 수정은 최소화되며, 기능 확장 시에도 구조를 안정적으로 유지할 수 있습니다.



핵심 코드 구현 1 - 델리게이트

UI 로그 출력 및 델리게이트 호출

BattleSystem.h

```
using OnBattleTextNumNumDelegate = std::function<void(std::string, int, int)>;
using OnBattleTextNumDelegate = std::function<void(std::string, int)>;
using OnBattleTextDelegate = std::function<void(std::string)>;

OnBattleTextNumNumDelegate OnHit;
OnBattleTextNumNumDelegate OnUseItem;
OnBattleTextDelegate OnAttack;
OnBattleTextDelegate OnDefeat;
```

전투 시스템에서 발생하는 주요 이벤트를 UI에 전달하기 위해 델리게이트를 선언, 호출한 부분입니다.
공격, 피격, 아이템 사용, 전투 종료를 각각 분리해 전달하도록 설계하여, BattleSystem은 전투 로직만 담당하고 출력은 별도 UI 모듈이 처리할 수 있도록 구성하였습니다.

```
if (OnUseItem) OnUseItem(Attacker->GetName(), 1, UseItem->GetItem().Value);

if (OnAttack) OnAttack(Attacker->GetName());

if (OnHit) OnHit(Defenser->GetName(), Damage, (int)StileMultiplier(Attacker, Defenser));

if (OnDefeat) OnDefeat(Enemy->GetName());
```

핵심 코드 구현 2.1 - 이미지 로드

PNG 이미지를 픽셀 데이터로 불러오고, 레이어를 합성한 뒤
ANSI 색상 코드와 블록 문자를 이용해 콘솔 텍스트 화면으로 변환했습니다.

ImageManager.h / ImageManager.cpp

콘솔 이미지 렌더링

```
void C_ImageManager::LoadToRaw(const std::string& name, const std::string& filePath) {
    int w, h, channels;
    unsigned char* data = stbi_load(filePath.c_str(), &w, &h, &channels, 4);


    if (data) {
        RawImage raw;
        raw.w = w; raw.h = h; raw.channels = 4;
        raw.data.assign(data, data + (w * h * 4));
        m_rawAssets[name] = raw;
        stbi_image_free(data);
    }
}
```

핵심 코드 구현 2.2 레이어 합성

PNG 이미지를 픽셀 데이터로 불러오고, 레이어를 합성한 뒤
ANSI 색상 코드와 블록 문자를 이용해 콘솔 텍스트 화면으로 변환했습니다.

ImageManager.h / ImageManager.cpp

콘솔 이미지 렌더링

```
float alpha = asset.data[srcIdx + 3] / 255.0f;

if (isBG) {
    for (int c = 0; c < 3; ++c)
        canvas[destIdx + c] = asset.data[srcIdx + c];
} else {
    for (int c = 0; c < 3; ++c) {
        canvas[destIdx + c] =
            (unsigned char)(asset.data[srcIdx + c] * alpha
            + canvas[destIdx + c] * (1.0f - alpha));
    }
}
```

핵심 코드 구현 2.3 텍스트 변환 및 ANSI 출력

PNG 이미지를 픽셀 데이터로 불러오고, 레이어를 합성한 뒤
ANSI 색상 코드와 블록 문자를 이용해 콘솔 텍스트 화면으로 변환했습니다.

ImageManager.h / ImageManager.cpp

콘솔 이미지 렌더링

```
for (int y = 0; y < maxH; y += 2) {
    int topIdx = (y * maxW + x) * 4;
    int botIdx = ((y + 1) * maxW + x) * 4;

    ss << "\x1b[38;2;" << fr << ";" << fg << ";" << fb << "m";
    ss << "\x1b[48;2;" << br << ";" << bg << ";" << bb << "m";
    ss << "▀";
}
```

핵심 코드 구현 3 – shared_ptr 기반 객체 관리

플레이어, 히로인, 아이템, 월드 객체는 여러 시스템에서 함께 참조해야 했기 때문에 shared_ptr를 사용해 객체 생명 주기와 소유권을 안정적으로 관리하되, UI및 로그 출력은 싱글톤으로 구현했습니다.

BattleSystem.h / BattleSystem.cpp

```
// P_Harem.cpp
shared_ptr<C_World> World = make_shared<C_World>();

// World.h
std::shared_ptr<C_Player> Player;
std::shared_ptr<C_Area> CurrentArea;
std::unordered_map<WorldArea, std::shared_ptr<C_Area>> Areas;
std::unique_ptr<C_BattleSystem> m_battle;

// Player.h
    std::vector<std::shared_ptr<C_Creature>> GirlFrends;
    std::shared_ptr<C_Inventory> Inventory;
```

게임 시작시 World를 하나 만들고 World가 각각의 Area와 BattleSystem, Player를 소유
Player가 Inventory와 GirlFrends들을 소유하는 구조

우리 팀만의 RPG, 그 이유

시스템 간의 연결 설계와 구조적 코드 작성을 통해 완성한 우리 팀 프로젝트의 핵심 성과와 차별점입니다.

유기적 시스템 설계

단순 기능 구현을 넘어, 호감도, 전투, 상성 시스템이 서로 연결되도록 설계해 게임의 개성 강화

- 호감도와 성장 시스템 연결
- 상성 기반 전투 전략 구현
- 히로인 중심 전투 구조 설계

구조 중심 구현

델리게이트와 기능별 폴더 구조를 활용해 로직 분리, 유지보수, 확장성을 함께 고려

- UI와 전투 로직 분리
- 기능별 폴더 구조 설계
- 시스템 간 결합도 최소화

효율적인 팀 협업

초기 구조와 역할을 먼저 정리하여 팀원별 분업과 병렬 개발이 가능하도록 만듦

- 담당 기능별 작업 분리
- 코드 충돌 최소화
- 구조 기반 협업 진행

